

T11.3 - Arquitectura Lambda - Almacenamiento y Master Dataset

Arquitectura Lambda

Almacenamiento - guía

Los grandes volúmenes de datos a almacenar son los **eventos**.

Definir de qué manera es conveniente almacenarlos depende de muchos aspectos a analizar.

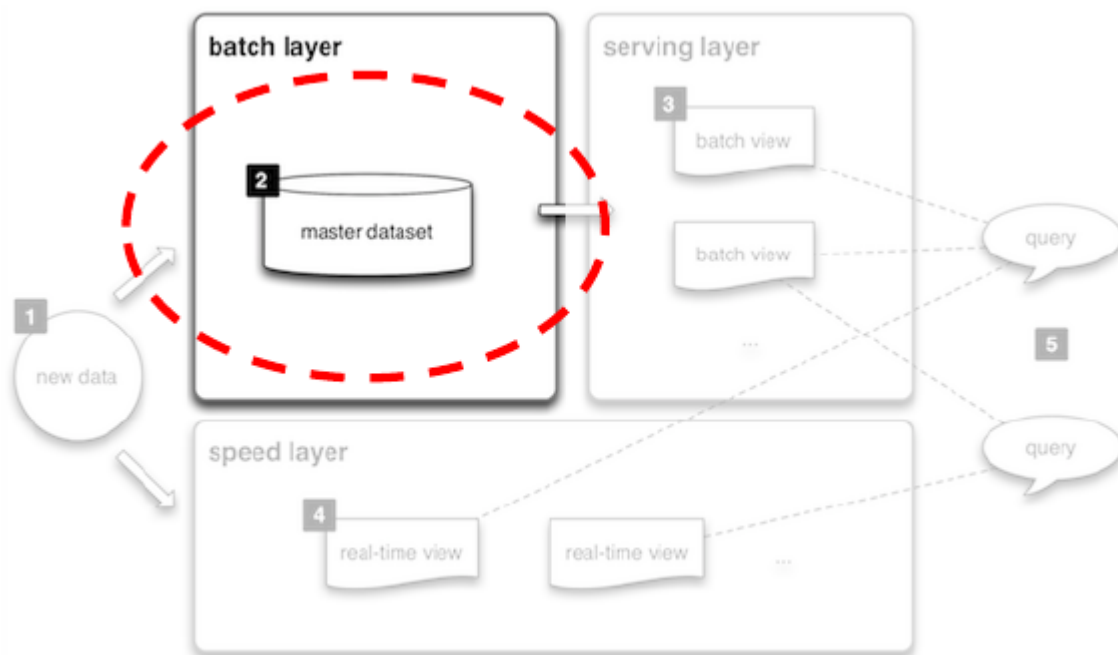
- ¿*Hasta dónde importa su estructura?*
 - Data Lake
 - Data Warehouse
- ¿*Qué uso le vamos a dar* a estos datos?
 - Fully Incremental
 - Event Sourcing
- ¿*Qué tipo de infraestructura es la más adecuada* para soportar los eventos?

Master Dataset

Es el *núcleo y fuente de la verdad* de la arquitectura lambda.

También es la única parte de la arquitectura lambda que debe estar más entre algodones, más cuidada.

Se puede acudir al master dataset para recuperar todo si se pierden todos los datasets de la serving y speed layer, porque las batch views de la serving layer se hacen en función del master dataset.

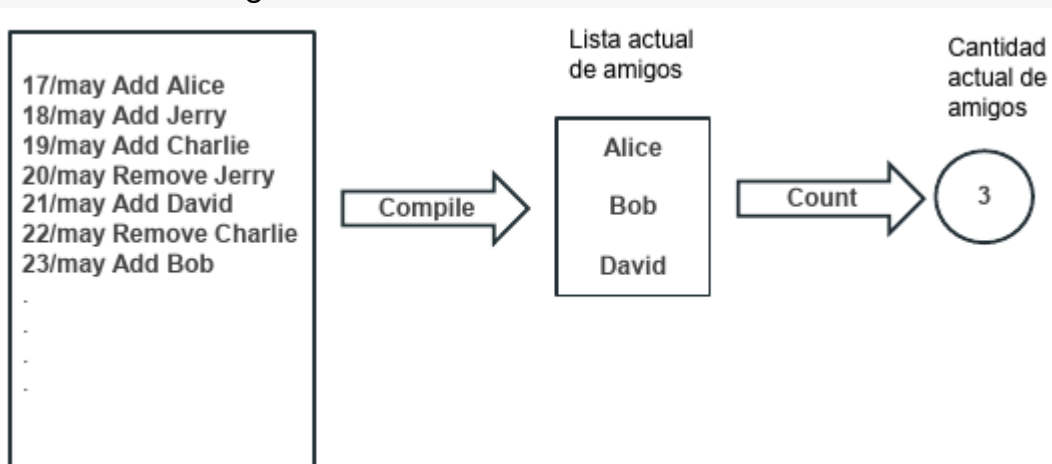


Propiedades de los datos (dependencia de la información)

Tom se suma a una red social. En ella, empieza a invitar a sus amigos y familiares.

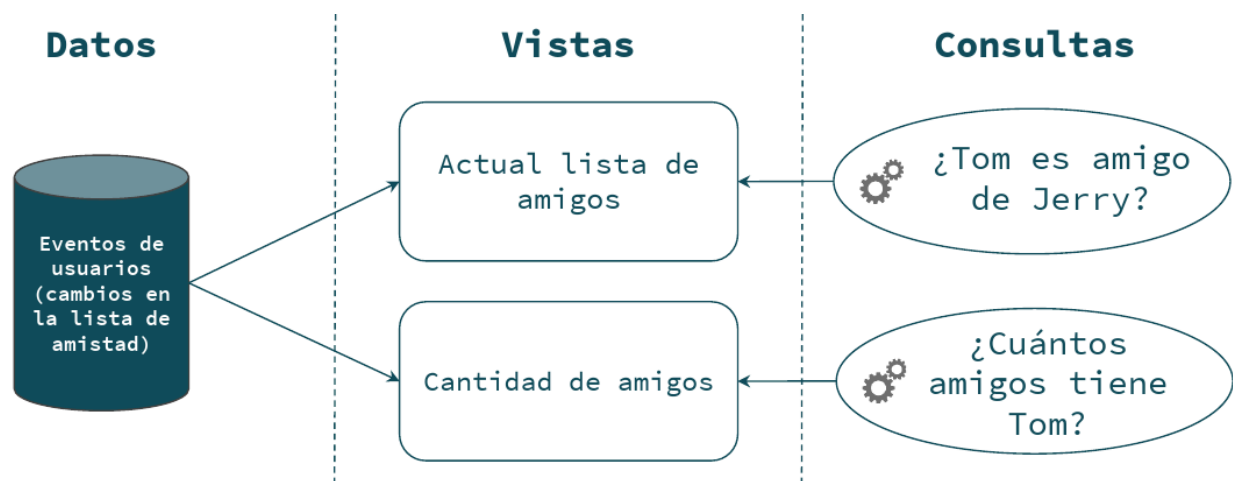
La información que se podría almacenar es:

1. Secuencia de eventos de las amistades de Tom y a quiénes borró.
2. Lista de amigos actual de Tom
3. Cantidad de amigos actual de Tom.



Dependencia de la información:

- **Datos** → premisas de *información que no pueden ser derivados* de otros elementos.
- **Información** → *conjunto total de datos* relevante al sistema.
- **Consultas (Queries)** → *preguntas* que se le hacen a la información.
- **Vistas (Views)** → información derivada de datos base *orientada a consultas*.



Propiedades (no funcionales) de los datos

Crudeza

Los datos deben ser tan crudos como sea posible.

Si son crudos, se puede derivar más información.



Menos crudos → más crudos.

Suelen ser *no estructurados antes que normalizados*.

¿Qué datos crudos almacenar? → Parsear los datos o una normalización semántica.

Una normalización semántica es reorganizar la información con formato libre en un formato estructurado de datos. Esto implica eliminar detalles del dato que puedan importar más adelante.

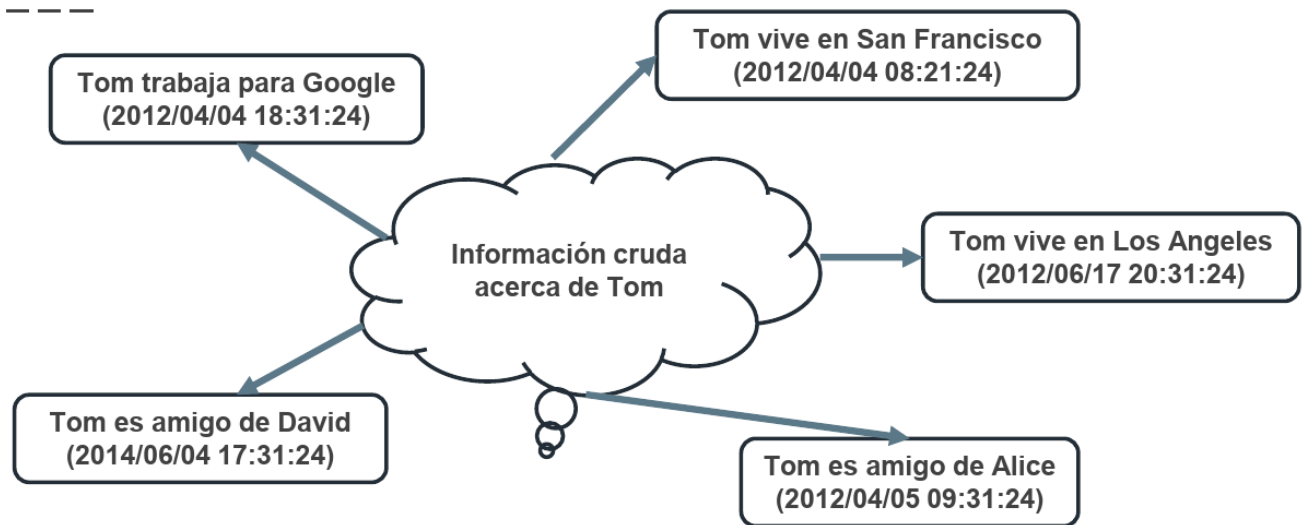
Inmutabilidad

- *No hay updates ni deletes*, sino que *sólo se agregan los datos*. Esto hace que el sistema tolere las fallas humanas → en caso de darse, se acude al master dataset.
- *Simplifica la carga de datos*.

Perpetuidad

- Los datos son *verdaderos para siempre* → la inmutabilidad no tendría sentido sin la perpetuidad.
- Se etiquetan los datos con *timestamps*.

Modelo basado en hechos



Se diseña un *esquema* como estos por cada tipo de evento a almacenar.

Cada *evento tiene un timestamp*.

Para diferenciar dos eventos idénticos se puede *agregar un valor aleatorio* en un campo adicional.

Esta técnica permite:

- Almacenar los datos como *hechos atómicos*.
- *Mantener estos hechos perpetuamente verdaderos* por haber incluido el timestamp.
- Evitar eventos *duplicados*.

Uso del master dataset

Típicamente será *demasiado grande para un único server* → hay que decidir cómo distribuir la data en múltiples máquinas.

La *forma de almacenarlo impacta en cómo se consume*.

Write once, read many.

Es un *patrón de acceso* que describe cómo se usa el master dataset.

- **Write once:** el master dataset es *immutable* → grandes volúmenes en *constante crecimiento*.
- **Read many:** casos de uso como ML, generación de batch views, etc. Generalmente *lecturas en bulk* (no random access) y *desde múltiples procesadores*.

La batch layer computa funciones sobre el dataset para producir batch views → necesita ser eficiente y estar optimizada a la hora de leer mucha data de una vez.

Operación	Requerimiento	Por qué
Escritura	Agregado (append) eficiente	Solo hay escrituras de nuevos eventos
	Almacenamiento escalable	Se guarda el dataset completo, la capacidad debe crecer en el tiempo
Lectura	Procesamiento paralelo	Construir la batch layer implica recorrer todo el master dataset

Infraestructura de almacenamiento distribuido

Key/value stores

Es la infraestructura de almacenamiento que se adapta mejor como batch layer storage.

La batch layer storage almacena el master dataset.

- Se piensa como un *hashmap gigante, persistente y distribuido* en muchas máquinas.
- Tiene muchas cosas *innecesarias* para el batch layer storage → no es la mejor opción para esta capa, aunque sí sirve para otras capas.

File system distribuido (mejor opción para batch layer)

- Los archivos son secuencias de bytes; la forma más eficiente de consumirlos es *escaneándolos completos*. Se almacenan secuencialmente en disco.
- Un filesystem estándar funciona en una única máquina, mientras que el *distribuido desparrama el almacenamiento en un cluster de computadoras*.
- Es *tolerante a fallos* → si se pierde una máquina, los archivos y datos siguen siendo accesibles.
- Operaciones más limitadas que un filesystem estándar → *no se puede escribir en el medio de un archivo ni modificarlo tras su creación*.
- *Ineficiente para archivos pequeños*.

¿Por qué se ajusta bien al master dataset?

- Orientado a *lectura secuencial*.
- Fácilmente comprimible.
- Sistemas robustos de *seguridad*.
- Es simple *enforzar inmutabilidad*.
- Organización jerárquica de directorios.

Formatos de serialización

¿Cómo se almacenan los datos dentro de un archivo?

- **Excel:** formato estándar y compacto, pero propietario y costoso de leer/escribir.

- **CSV:** rápido de leer/escribir, poco compacto y con problemas de caracteres especiales.

En la práctica se usan *formatos de serialización binarios* que permiten:

- *Definir esquemas* (para producir, validar e interpretar mensajes).
- *Optimizar el uso de espacio.*
- *Optimizar la lectura.*

Opciones no tan simples (ambos binarios, permiten comunicación entre procesos en distintas tecnologías):

- **Protocol Buffers.**
- **Thrift.**